Deep Learning for Economics and Finance

Agentic Programming: Using AI Agents for Research

Simon Scheidegger

HEC, University of Lausanne

Based on: Goldsmith-Pinkham (2026); Bilionis et al. (2026); Korinek (2023, *JEL*); Korinek & Balwit (2023, *AEA P&P*); Schluntz & Zhang (2024, "Building Effective Agents"); Cherny (2025), "10 Claude Code Tips"; Anthropic Docs (2026).

Materials:

https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models

From Building Models to Shipping Workflows

The earlier lectures built the models.

- Deep-learning fundamentals
- Deep Equilibrium Nets (DEQNs)
- IRBC, NAS, loss normalization
- Autodiff, sequence space, OLG

Each notebook ran for hours. Most of that time was *not* modeling: it was wiring data, debugging shapes, refactoring loss terms, testing on toy problems, and rebuilding figures.

This lecture builds the workflow.

- Set up Claude Code for research projects
- Drive it with Level-3 prompts
- Codify your conventions (`CLAUDE.md`, skills, subagents, hooks)
- Close the loop with Ralph for spec-driven autonomy

Today's promise

By the end of class you will have run an autonomous loop that implements a small estimator, runs its tests, and commits — with you only specifying what "done" looks like.

Why Use Agentic Programming?

Speed

- Turn vague ideas into runnable code quickly
- Compress debug and refactor cycles from hours to minutes
- Keep momentum during exploratory research

Scope

- One workflow can span data, code, figures, tables, and LaTeX
- Easy to move from prototype to reproducible pipeline
- Strong fit for replication and robustness checks

Leverage

- Your comparative advantage shifts from typing to specification
- Domain knowledge matters more, not less
- Best used as a force multiplier, not an autopilot

Bottom line

Agentic programming is valuable because it lets researchers iterate faster, cover more of the workflow, and spend more time on identification, interpretation, and verification.

What Is Agentic Programming?

Working definition

Agentic programming means working with an AI system that can:

- read project files and relevant context
- propose and edit code or text
- run commands, inspect outputs, and debug
- iterate toward a goal instead of answering once

Core loop

Plan → **Act** → **Observe** → **Revise**

What makes it "agentic"

- Tool use
- Multi-step execution
- Feedback from the environment
- Partial autonomy within constraints

What it is not

- Not just autocomplete
- Not one-shot code generation
- Not autopilot for research judgment

A Concrete Economics and Finance Workflow

Task: "Replicate Table 2 and Figure 3 from a paper using a panel dataset, then draft the robustness paragraph in the paper's style."

1. Inspect the data and write a diagnostics memo
2. Build or fix the cleaning and estimation scripts
3. Produce the regression table and figure
4. Export the outputs into LaTeX-ready files
5. Draft a short results paragraph tied to the evidence
6. Run verification checks before anything is trusted

Inputs you provide

- Research question
- Data description
- Method constraints
- Output format
- Verification rules

Deliverable

One supervised workflow can produce code, tables, figures, LaTeX, and notes in the same session.

Who This Is For and What You'll Learn

Audience

- PhD students
- Researchers in economics and finance
- Computational social scientists

Prerequisites

- Basic Python familiarity
- Laptop with internet access
- No prior AI-tooling experience required

By the end of the lecture, you should be able to

- set up a safe, version-controlled agentic workflow
- write research prompts that specify task, context, and verification
- use `CLAUDE.md`, skills, and subagents to structure work
- automate a mini pipeline from data to figures to LaTeX
- use agents for replication without outsourcing judgment

Roadmap of This Lecture

Pre-flight: run Exercise 0 (`exercise_prompts.md`) once before class.

Block 1: Foundations (75 min, Parts I–IV)

- Orientation & mental models (15 min)
- Environment setup + **Ex. 1** (20 min)
- Core interaction + **Ex. 2** (20 min)
- Prompt engineering + **Ex. 3** (20 min)

— end of Block 1 —

Block 2: Advanced & Hands-On (Parts V–IX)

- CLAUDE.md + **Ex. 4** (15 min)
- Skills, subagents & hooks + **Ex. 5** (15 min)
- Autonomous loops (Ralph) + **Ex. 12** (15 min)
- Data → paper + replication + **Ex. 6–7** (15 min)
- Verification, ethics & wrap-up (15 min)

Today's Goal

Go from zero to a production-grade AI-assisted research workflow:

- Set up Claude Code with CLAUDE.md
- Write Level 3 research prompts
- Create custom skills, subagents, hooks
- Drive an autonomous loop with Ralph
- Attempt a paper replication

Prerequisites

- Laptop with internet access
- Basic Python familiarity
- No prior AI-tooling experience needed

Part I

Orientation & Mental Models

15 minutes

Why This Matters Now

The bottleneck in computational research has shifted:

- **Old bottleneck:** Can you write the code?
- **New bottleneck:** Can you correctly specify what you want?

This rewards domain expertise. Researchers in economics and finance have a natural advantage over pure software engineers, because what is hard is knowing *what question to ask*.

Two traps to avoid:

1. **Hype trap:** "AI wrote a paper in 30 min!" Technically possible; quality gap is real.
2. **Dismissal trap:** "This is just autocomplete." Wrong—modern agents autonomously plan, execute, debug, and iterate across multi-hour sessions.

Before 2023
Expertise + coding skill

↓

After 2023
Expertise + specification skill
(AI handles implementation)

Your economics and finance training is the competitive advantage

Key Insight

All AI coding tools use the same underlying models. The difference is scaffolding—how much environment access the agent has—not raw intelligence.

The AI Landscape: Three Major Players

Snapshot as of early 2026 • semi-subjective • re-check before re-using.
Frontier models, pricing, and rankings change on a monthly cadence. Treat this slide as a working opinion, not a benchmark.

ChatGPT *OpenAI*

- Strong math & analytical reasoning
- Broadest ecosystem (voice, GPTs, Codex)
- Default general-purpose assistant

Claude *Anthropic*

- Excellent writing, editing, close reading
- Very strong coding and project work
- Mature terminal agent via `claude code`

Gemini *Google*

- Strong on long files and multimodal input
- Tight Google ecosystem integration
- Good if you live in Docs/Drive/Gmail

Task	ChatGPT / Codex	Claude	Gemini
Math & analytical reasoning	✓✓	✓✓	✓
Writing & editing	✓	✓✓	✓
Coding & repo-level work	✓	✓✓	✓
Long files / large context	✓	✓✓	✓✓
Google ecosystem tasks	—	—	✓✓

Why Claude Code for this course: all three vendors ship comparable browser chatbots; the distinctive piece is the terminal agent. Anthropic's `claude code` is the most mature in early 2026. Most concepts (skills, subagents, hooks, MCP) transfer to OpenAI Codex CLI, Gemini CLI, Cursor agent mode, or Aider with minor syntax changes; pick the leaderboard *(SWE-Bench Verified, Aider polyglot, WebDev Arena)* benchmark you trust and re-check quarterly.

Other notable players we are not covering: **xAI Grok, Meta Llama, Mistral, DeepSeek, Qwen — strong on specific axes (open weights, on-device, Chinese-language) but less mature for research-grade terminal agents at the time of writing.**

Chatbox vs. Agentic: A Useful Analogy

Paul Goldsmith-Pinkham (2026)

"Web-based AI is like exchanging emails with a smart person. Claude Code is like having them sit next to you, coding while you talk."

Think *browser chat*

- Ideation, explanations, feedback
- Task mostly in your head
- Usually copy–paste

Work With Files *desktop app*

- Drop in PDFs, screenshots, spreadsheets
- Read & draft around single files
- Still outside your project tree

Run Workflows *terminal agent*

- Read, write, and execute code
- Best when reproducibility matters
- Treat it like a fast junior RA

Today's goal: get every one of you from browser-chatbot copy-paste to a terminal agent working inside a real, version-controlled research project. The next slide maps that progression onto five discrete levels.

The Hierarchy of AI Coding Tools

Level	Capability	Examples (early 2026)
0–1	Browser chatbot + copy-paste	ChatGPT, Claude.ai, Gemini
2	Inline IDE assistant	Copilot (autocomplete), Cursor (composer)
3	**Terminal agent / agent mode—full env. access**	**Claude Code**, Codex CLI, Cursor agent, Aider, Copilot agent
4	Level 3 + external tools via MCP	+ Zotero, GitHub, HPC, Brave Search
5	Closed-loop autonomy with verifier (1–2 hrs)	**Ralph** loops, Cursor Background Agents

What makes Level 3+ different from Level 0–2?

- Reads and writes files on your machine
- Executes code and sees the output
- Uses git: commit, branch, diff
- Installs packages, manages environments
- Iterates autonomously—fixes its own errors

This Lecture

We get you to **Level 3–4**, with a taste of Level 5 (Ralph). The jump from L2 to L3 is the largest productivity gain: L2 suggests edits, L3 does the work.

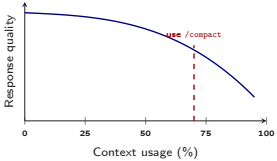
The Context Window — The Most Important Mental Model

An LLM (large language model) has no persistent memory. Every message bundles the **entire conversation history**—your prompts, Claude's responses, every file read, every code output—into one large document sent to the model.

This is the context window:

≈ 200K tokens ≈ 500 pages

The performance curve: Quality degrades as context fills. A session at turn 5 is sharper than the same session at turn 35.



Practical implications (from Anthropic docs):

1. Break work into focused sessions
2. Write state to disk (`progress.md`)
3. Use `/compact` before overflow
4. Fresh session (`/clear`) per sub-task
5. Subagents for research keep main context clean

Boris Cherny: "**Context management is the single most critical resource.**"

Where AI Excels vs. Struggles

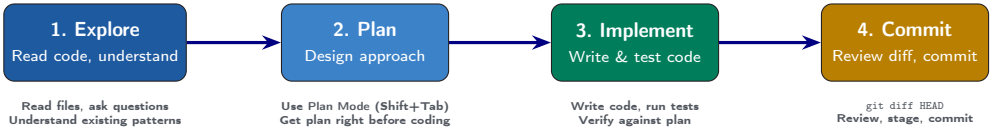
Task Type	AI Quality	Notes
Data cleaning, reshaping	**Excellent**	Very fast ROI
Standard econometrics (OLS, GMM)	**Excellent**	Verify edge cases
Debugging cryptic errors	**Excellent**	Faster than Stack Overflow
Boilerplate (LaTeX, README)	**Excellent**	
Data analytics (SQL, dashboards)	**Excellent**	Use with CLI or MCP
Novel theory derivation	**Poor**	Never trust without checking
Methods not in training data	**Moderate**	Degrades gracefully
Verification / self-checking	**Poor**	Never skip human review
Subtle identification issues	**Poor**	Economist must lead

Key Insight (Bilionis replication experiment, 2026)

AI is a very fast second-year PhD student: excellent at known methods, mediocre at novel ones. Succeeded on in-distribution (SINDy), failed on out-of-distribution (unpublished BNN).

The Master Workflow: Explore → Plan → Implement → Commit

This four-phase workflow is the single most important pattern from the Anthropic docs and Boris Cherny's team. It applies to every non-trivial task.



TIP 2 **Pour energy into the plan** so Claude can one-shot the implementation. If something goes sideways, switch *back* to plan mode and re-plan instead of pushing through.

Skip planning only for:

- Single-line fixes (typos, obvious bugs)
- Adding a single function with clear requirements
- Tasks where you gave very specific instructions

Part II

Production Environment Setup

20 minutes (includes Exercise 1)

The Core Stack DEMO

You need four things: a good terminal, Claude Code, version-control discipline, and Python.

Install (macOS / Linux)

```
# Terminal (macOS: brew install ghostty)
# Claude Code
npm install -g @anthropic-ai/claude-code
# Python package manager
curl -LsSf https://astral.sh/uv/install.sh
      | sh
```

TIP 7 **Terminal setup tips:**

- Use `/statusline` to show context usage and git branch
- Color-code terminal tabs by session purpose
- Voice dictation (fn×2 on macOS)—you speak 3× faster than you type
- Use Zellij split panes: Claude | Editor | Output

SELF-STUDY

Exercise 0 "Hello Claude" — What Success Looks Like

Run before class (or before reading further if self-study). Confirms install, auth, and the basic "read → write a file" loop in one minute.

1. Verify the binary

```
$ claude --version
1.x.y (Claude Code)
```

2. Scratch folder + launch

```
$ mkdir -p ~/research/hello-claude
$ cd ~/research/hello-claude
$ echo "Just testing." > notes.txt
$ claude
[Welcome to Claude Code]
```

3. Paste this prompt

```
> What files are in this directory?
  Then read notes.txt and rewrite
  it as a one-sentence haiku.
  Save to haiku.md.
```

4. Expected response (paraphrased)

```
I see notes.txt. Reading it...
"Just testing." -- writing haiku to haiku.md.
Done. Run `cat haiku.md` to see it.
```

Troubleshooting: *not found* ⇒ `npm install -g @anthropic-ai/claude-code`; re-open terminal. *auth prompt* ⇒ run `claude` again, browser opens for OAuth.

Claude Desktop: Chat, Cowork, Code

Before the terminal agent, know the three modes inside the `claude.ai` desktop app. You will use all three during a typical research day.

Chat

- Standard conversational interface
- Brainstorm, summarize, explain
- Upload PDFs, images
- Same as `claude.ai` in the browser

Use for: ideation before you touch any code.

Cowork

- "Claude Code for non-developers"
- Agentic: *executes* tasks, not just discusses them
- Organises files, analyzes spreadsheets, generates reports
- Can operate desktop apps on your behalf

Use for: admin, light data work without a terminal.

Code

- The terminal agent (`claude code`)
- Full file-system access
- Reads, writes, and runs code
- Most powerful mode for research projects

Use for: everything reproducible.

Model selection (all three modes)

Switch between **Opus** (smartest, higher token cost) and **Sonnet** (fast, cheaper, excellent for most tasks) with `/model` mid-session. Use **Haiku** for cheap verifier subagents. Same models across Chat, Cowork, and Code.

Claude Code: First Launch & Key Commands

```
# Launch in your project
cd ~/research/my-project
claude

# Bootstrap project memory
/init

# Check your setup
/model
/cost
```

Subscription tiers (2026):

Tier	Price	Use case
Pro	$20/mo	Getting started
Max 5x	$100/mo	Daily research
Max 20x	$200/mo	Multi-session, autonomous
Team	$30/seat	Lab / co-author group
Enterprise	custom	SSO, ZDR, audit logs
API	$15–75/MTok	Pay-per-use

Essential commands:

Command	Purpose
`/help`	List all commands
`/model`	Switch model mid-session
`/cost`	Token spend so far
`/compact`	Compress conversation
`/clear`	Fresh context
`/init`	Bootstrap CLAUDE.md
`/agents`	Manage subagents
`/permissions`	Tool-allow rules
`Ctrl+C`	Interrupt execution
`ESC`	Cancel / undo turn
`ESC`×2	Rewind to checkpoint
`Shift+Tab`	Toggle Plan Mode

Privacy

Files stay local, but anything Claude reads enters the API context. Protect sensitive files with a `CLAUDE.md` rule ("never read `data/raw/`") plus a `PreToolUse` hook (Part VI) that hard-blocks the path.

Git: Your Safety Net

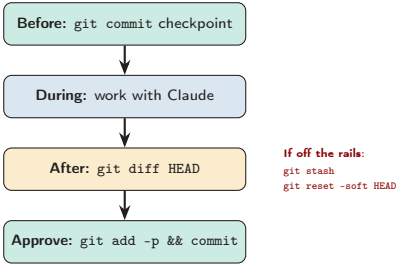
AI-generated changes that are not version-controlled cannot be rolled back. Git is non-negotiable.

```
# Create a research project
mkdir ~/research/my-paper
cd ~/research/my-paper
git init && git branch -M main

# Protect sensitive paths via CLAUDE.md
# (and a PreToolUse hook -- see Part VI)
cat > CLAUDE.md << 'EOF'
## Read-only paths
- data/raw/        # raw sensitive data
- .env             # API keys
- credentials/
EOF

# Create structure + first commit
mkdir -p {data/{raw,processed},
          code,figures,paper,notes}
git add . && git commit -m "initial"
```

The Git discipline cycle:



Before every session: commit. After every session: review the diff. Takes 30 seconds; makes everything recoverable.

MCP: Connecting Claude to External Tools

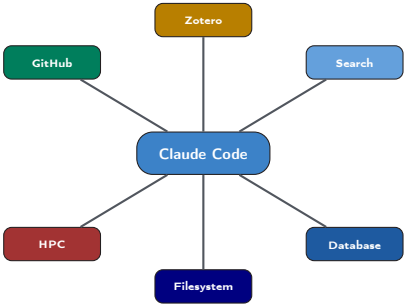
Model Context Protocol (MCP) connects Claude to external services, turning it from Level 3 to Level 4.

```
# Install MCP servers
claude mcp add zotero        # references
claude mcp add github        # PRs, issues
claude mcp add brave-search  # web search
claude mcp add filesystem    # extended I/O
```

Usage in a session:

```
> Search Zotero for papers on
  carbon taxation in OLG models
> Create a GitHub branch called
  feature/robustness-checks
```

TIP 9 Use Claude with the bq CLI (or any database CLI/MCP) to pull and analyze metrics. Boris says he hasn't written SQL in 6+ months.



Each MCP server runs locally and communicates via JSON-RPC. Claude calls tools through the server; your data never leaves your machine except through the API call itself.

Exercise 1: Environment Bootstrap (15 min)

Goal

Get a working Claude Code session in a version-controlled project.

1. Create directory: `mkdir ~/research/lecture-demo`
2. Copy `day5/code/generate_synthetic_data.py` and run it once
3. Initialize git: `git init && git branch -M main`
4. Launch Claude: `claude`
5. Run `/init` to bootstrap CLAUDE.md
6. Type: `> What files are in this directory?`
7. Type: `> Create explore_data.py for data/synthetic_panel.csv. Use pathlib.`
8. Inspect the file. Run it on `data/synthetic_panel.csv`.
9. `git add . && git commit -m "exercise 1"`

Success criteria:

- ✓ Claude Code running
- ✓ Git repo with ≥ 1 commit
- ✓ CLAUDE.md exists
- ✓ Python file runs

Troubleshooting:

- `claude: not found` → reinstall
- Auth error → `claude auth`

See `day5/code/exercise_prompts.md` for copy-pasteable prompts.

SELF-STUDY

Exercise 1: A Full Worked Session

What a healthy first run looks like, end to end. Blue = you; green = Claude.

Turn 1 – you

```
> What files are in this directory?
  Summarize the structure.
```

Turn 2 – Claude

```
[Listed: data/synthetic_panel.csv
 (5000 x 6: firm_id, year, revenue,
  employees, treated, post)]
One CSV: balanced panel of 500
firms x 10 years; 2015 treatment.
No code yet.
```

Turn 3 – you

```
> Create explore_data.py: takes a
  CSV path on the CLI; prints shape,
  dtypes, first 5 rows, summary
  stats, % missing. Use pathlib.
  Add __main__ guard.
```

Turn 4 – Claude

```
[Wrote explore_data.py]
[Ran python3 explore_data.py
        data/synthetic_panel.csv]
shape (5000, 6)  missing 0/6
revenue mean=10.32  sd=0.45
First 5 rows: [...]
```

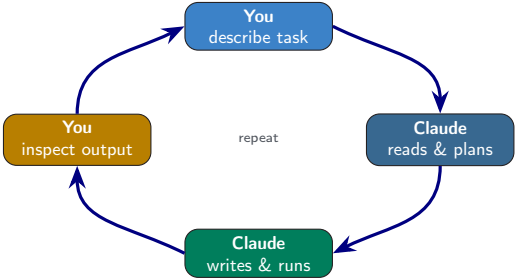
What you should observe. **Claude (a) inspected before writing, (b) satisfied every spec item in one pass, (c) ran the script itself.** ***If your session looks like this, your environment is healthy — ready for Ex 2.***

Part III

The Core Interaction Loop

20 minutes (includes Exercise 2)

The Core Feedback Loop DEMO



The most important skill: how to interrupt.

`Ctrl+C`	Stop execution
`ESC` ×1	Cancel current message
`ESC` ×2	Rewind to checkpoint
`/rewind`	Go back N turns

Do not let Claude finish a bad path and then course-correct. Interrupt early. After two failed corrections, `/clear` and write a better prompt from scratch.

TIP 5 **Claude fixes most bugs by itself.** Paste a Slack bug thread and say "fix." Or say "Go fix the failing CI tests." Don't micromanage *how*.

SELF-STUDY

The Feedback Loop in Action

One iteration condensed. Blue = you; green = Claude; orange = you steering.

1. You describe the task

```
> data/synthetic_panel.csv has firm-year revenue.
  Print the sample mean per year. Use pandas. No plot yet.
```

2. Claude reads, writes, runs

```
[Reads synthetic_panel.csv -- 5000x6]  [Writes scripts/year_means.py]
[Runs python3 scripts/year_means.py]
2010  9.95  2012 10.18  2014 10.31  2016 10.45  2018 10.55
2011 10.02  2013 10.25  2015 10.38  2017 10.50  2019 10.62
```

3. You inspect, catch a bug, steer

```
> The 2015 jump looks suspicious -- is revenue already in logs?
  Print the column dtype and first 3 rows before the means.
```

Lesson: steer between turns, not after a long bad path. Had turn 3 been "great, now make a 4-panel figure," the bug from turn 2 would have silently propagated.

Prompt Quality Spectrum

Every research prompt exists on a spectrum from vague to precise. Level 3 is always the goal.

Level	Type	Example
0	Vague	`> Analyze this data`
1	Specific	`> Load gdp.csv. Compute YoY growth rates.`
2	+ Context	`> Column 'gdp_real' is in 2015 USD billions, quarterly.`
3	**+ Constraints + Output + Verify**	`> Use .iloc not .ix. Write to processed/. Output: Markdown table. Flag outliers >10%. Assert N > 1000.`

What makes Level 3 different:

- Exact file paths and column names
- Formula or method specified explicitly
- Negative constraints (what NOT to do)
- Output format defined
- Verification step included

TIP 6 **Challenge Claude:**

- "Grill me on these changes and don't make a PR until I pass your test."
- "Knowing everything you know now, scrap this and implement the elegant solution."
- Specificity is not nagging—it is the entire skill.

A Level 3 Prompt in Detail DEMO

```
> Load data/gdp_quarterly.csv.
  - Column 'gdp_real': real GDP in 2015 USD billions, quarterly.
  - Compute year-over-year growth: g_t = (gdp_t / gdp_{t-4}) - 1
  - Print a summary table with: mean, sd, p5, p95, N
  - Flag quarters where |g_t| > 0.1 as outliers, print a list
  - Write the cleaned series to data/processed/gdp_growth.csv
  - Do NOT use pandas deprecated methods -- use .iloc, not .ix
  Expected output format: Markdown table printed to stdout.
```

Checklist—what makes this Level 3:

- ✓ Exact file path & column name
- ✓ Formula specified (g_t)
- ✓ Output statistics listed
- ✓ Outlier threshold given
- ✓ Output path specified
- ✓ Negative constraint (no deprecated API)
- ✓ Expected output format

Common research task patterns:

A "Explain what this code does"

B "Write code matching existing style"

C "Debug this error"

D "Refactor for reproducibility"

E "Generate a LaTeX table"

See exercise handout for full templates of each pattern.

SELF-STUDY

Claude's Response to the Level-3 Prompt

After ∼40 seconds Claude reads the file, writes the script, runs it, prints the summary, and persists the cleaned series.

Terminal output (abridged)

```
[Read gdp_quarterly.csv -- 320 rows]
[Wrote scripts/gdp_growth.py]
[Ran python3 scripts/gdp_growth.py]

stat    mean     sd      p5      p95    N
value   0.0205   0.018  -0.008   0.049  316

Outliers (|g_t| > 0.10):
  2008Q4:  -0.121
  2020Q2:  -0.219
  2020Q3:   0.156

[Wrote processed/gdp_growth.csv]
```

End to end, in one pass:

1. Read the raw CSV; inspected schema.
2. Wrote `scripts/gdp_growth.py`.
3. Ran it; saw the table.
4. Persisted the cleaned series.
5. Surfaced three outliers at your threshold.

Why this is Level 3

No re-prompt for the table, no re-prompt for the file write, no re-prompt for outliers. Each was *specified up-front*; Claude satisfied all six checklist items in one pass.

Parallel Sessions & Worktrees

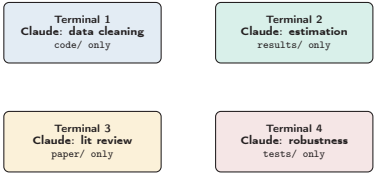
TIP 1 **Do more in parallel.** This is the single biggest productivity unlock from the Claude Code team.

Spin up 3–5 git worktrees, each running its own Claude session:

```
# Create worktrees
git worktree add ../proj-a -b data-clean
git worktree add ../proj-b -b estimation
git worktree add ../proj-c -b lit-review

# Launch Claude in each
cd ../proj-a && claude  # Terminal 1
cd ../proj-b && claude  # Terminal 2
cd ../proj-c && claude  # Terminal 3
```

Worktrees share repo state (unlike clones), reducing merge conflicts. Each session works on non-overlapping files.



Key: Set scope explicitly in each session:

```
> You are only allowed to modify
  files in the code/ directory.
```

Cost warning: 3–5 sessions consume API quota in parallel. On Pro ($20/mo) you will hit limits within minutes; Max or API pay-as-you-go is the realistic tier here.

Context Management in Practice

The progress file pattern:

For any session > 10 turns.

```
> Write notes/session_state.md:
  1. What we have done so far
  2. What is working
  3. What we need to do next
  4. Design decisions and why
```

Next session:

```
> Read notes/session_state.md and
  continue from where we left off.
```

Session resume options:

- `claude -continue` — resume last session
- `claude -resume` — pick from recent sessions
- `/rename "estimation pipeline"` — name sessions for easy recall

When to use which approach:

Situation	Action
< 5 turns	Just work directly
5–15 turns	Write progress file at end
> 15 turns	`/compact` + progress file
New sub-task	`/clear` or new session
Research task	Use subagent to keep main context clean

Anthropic's #1 Failure Pattern

Kitchen-sink sessions: mixing unrelated tasks in one session. Each task adds context that degrades all other tasks. Use `/clear` between unrelated work.

Exercise 2: First Real Task (15 min)

Goal

Use a Level 3 prompt on real data and practice context management.

```
> I have a dataset at data/synthetic_panel.csv.
  Without loading it yet, first tell me how to inspect it.
  Then:
  1. Load it and print diagnostics (columns, shape, date range, missing)
  2. Compute annual average revenue by treatment group
  3. Generate a 2x2 panel of plots (matplotlib,
     publication-ready, no gridlines, serif font, 6x5 inches)
  4. Save the figure to figures/panel_overview.pdf
  5. Write a 3-sentence description to notes/data_description.md
```

No CSV? First run `python generate_synthetic_data.py` in the project root.

Reflection: Where did Claude do well? Where did it need correction? At what point would you have interrupted? What constraint would have prevented any issues?

Part IV

Prompt Engineering for Research

20 minutes (includes Exercise 3)

Anatomy of a Research Prompt

Every good research prompt has up to six components. Not every prompt needs all six—but omitting any one produces a specific failure mode.

#	Component	What it does	Failure if missing
1	**ROLE**	Who Claude is in this task	Generic, non-expert output
2	**CONTEXT**	What it needs to know	Asks too many questions
3	**TASK**	What you want done	Invents a different task
4	**CONSTRAINTS**	Must / must not do	"AI's code" not "my code"
5	**OUTPUT**	Deliverable format	Wrong format, extra junk
6	**VERIFICATION**	How to check it worked	Plausible but wrong

The Meta-Rule

When something goes wrong, ask: "Which of the 6 components did I omit?" The answer is almost always the fix.

The Constraint Vocabulary

Constraints are the difference between "AI wrote the code" and "AI wrote my code."

Negative (what NOT to do):

```
Do NOT use statsmodels
  -- use linearmodels.
Do NOT change existing
  function signatures.
Do NOT use deprecated
  pandas syntax.
Do NOT hardcode file
  paths.
```

Positive (what to do):

```
Always use type hints.
Follow AEA data policy:
  reproducible from one
  script.
Use exact notation from
  paper/main.tex.
Maintain backward
  compatibility with
  test suite.
```

Scope (boundaries):

```
Only modify files in
  the code/ directory.
Do not touch paper/
  or data/raw/.
Make the minimum change
  needed to fix the bug.
Do one thing at a time;
  stop and report before
  the next step.
```

The "Think Before You Code" Pattern

For complex tasks, force a planning step: "BEFORE writing any code: (1) Describe your plan, (2) List every file you'll modify, (3) List risks. Do not proceed until I say 'go ahead.'" This single pattern prevents ∼80% of "Claude rewrote everything" disasters.

Research Prompt Templates

Template 1: Replication Check

```
> Read code/estimate.py and
  paper/main.tex Sec 3.2--3.4.
  Check: does the code match
  the model in the paper?
  Write a discrepancy report:
  ✓ [item]: matches
  × [item]: paper says X, code does Y
  ? [item]: unclear from paper
```

Template 2: Robustness Table

```
> Add 3 robustness checks to Table 2.
  Baseline: code/estimate_baseline.py
  Specs: (1) add year FE
         (2) drop leverage > 0.9
         (3) winsorize at 1%/99%
  For each: separate script, save to
    results/robustness_{name}.json
  Then generate a LaTeX table with
  all 4 specs side by side (booktabs).
```

Template 3: Autonomous Session

```
> I'm stepping away for 30 minutes.
  Task: implement the pipeline
  per plan.md.
  Rules while I am away:
  - Commit after each major step
  - If stuck after 3 attempts,
    write to notes/blocked.md
    and STOP
  - Do NOT modify data/raw/
  - Write progress every 10 steps
    to notes/progress.md
  When done: summary to
    notes/session_end.md.
```

Template 4: Literature Writing

```
> Do NOT invent any citations.
  Only cite papers already in
  paper/references.bib.
  If not in bib, write:
  [CITE: author, year -- verify]
```

Common Failure Modes and Fixes

Failure	Cause	Fix
Claude "invents" results	Vague output spec	Add explicit verification step
Code works but unreadable	No style constraint	Specify style in role prompt
Modifies wrong files	No scope constraint	Add explicit file scope
Long session degrades	Context overflow	`/compact`, break into sessions
Loops on same error	No retry limit	"If 3 attempts fail, stop"
Wrong statistical method	Insufficient domain	Role prompt + method name
Output $\neq$ paper	Disconnected context	Point Claude to paper section
Over-specified CLAUDE.md	Too-long instructions	Keep CLAUDE.md $<$ 200 lines

Top 3 anti-patterns (Anthropic docs):

1. **Kitchen-sink sessions:** mixing unrelated tasks
2. **Correcting repeatedly** instead of starting fresh
3. **Trust-then-verify gap:** no tests or expected outputs

Highest-leverage fix

Give Claude a way to verify its own work. Tests, expected outputs, or synthetic data with known answers prevent most failures.

Exercise 3: Prompt Refactor (15 min)

Goal

Take a bad prompt and improve it using the 6-component framework.

Starting prompt (Level 0—everything is wrong):

```
> Analyze the data and run the regression and make a table.
```

Your task:

1. **Diagnose:** Which of the 6 components are missing? (Answer: all of them.)
2. **Rewrite:** Create a Level 3 version for your own research context.
3. **Peer review:** Share with a neighbor and critique each other's prompts.
4. **Test:** Run both versions and compare output quality.

Bonus: Take a prompt you've used in the past that produced disappointing results. Apply the 6-component framework and rerun it.

Part V

Project Memory & CLAUDE.md

15 minutes (includes Exercise 4)

What is CLAUDE.md?

`CLAUDE.md` is read automatically at session start—your project's persistent memory for a capable but amnesiac research assistant.

Where: `~/.claude/CLAUDE.md` (user-global), `./CLAUDE.md` or `./.claude/CLAUDE.md` (project root), `subdir/CLAUDE.md` (path-scoped). All are loaded; nearest wins. Use `@path/to/file.md` to import shared rules across projects.

Highest-value sections:

1. Repository structure (read-only dirs)
2. Coding standards
3. Statistical conventions & notation
4. Current status (update each session)
5. "Do Not Touch" list

TIP 3 **Invest in your CLAUDE.md.** After every correction:

> *"Update CLAUDE.md so you don't make that mistake again."*

Claude is eerily good at writing rules for itself. Keep iterating until mistake rate drops.

Best practices (Reddit + Anthropic):

- Keep root CLAUDE.md <200 lines
- Specific rules in subfolder files
- Use `.claude/rules/` for path-scoped rules
- Critical rules: use hooks (code enforcement)

CLAUDE.md Template for Economics Research

```
# CLAUDE.md -- [Paper Title]

## Project Overview
- **Topic:** [one sentence]
- **Stage:** draft / revision / final
- **Target journal:** [e.g., RES]

## Repository Structure
data/
  raw/        <- READ ONLY. Never modify.
  processed/  <- Your outputs go here
code/
  estimate.py <- Main estimation
  config.py   <- All parameters here
figures/      <- PDF format
paper/main.tex

## Coding Standards
- Python 3.11+; type hints on all sigs
- pathlib.Path for all file I/O
- Seeds: set in config.py, pass explicit
- PEP 8, max line length 88 (black)

## Statistical Conventions
- SEs: clustered at [firm] level
- FE: two-way (firm + year) baseline

## Notation (matches paper/main.tex)
- i = firm, t = year
- Y_it = outcome variable
- D_it = treatment indicator
- beta = coefficient of interest

## What I Am Working On Now
[UPDATE THIS EVERY SESSION]
- Current task: ...
- Last completed: ...
- Next step: ...

## Do Not Touch
- data/raw/ -- needs confirmation
- Main estimation function signature
- Any file in paper/ without asking

## Known Pitfalls
- [AI mistakes to avoid -- add here]

## Decisions Made (and Why)
- Used OLS not IV because [reason]
```

Copy-pasteable version: day5/code/CLAUDE_md_template.md

Three-Layer Architecture for AI-Assisted Projects

From the course's `utils-agentic-support` framework:

Layer	Purpose	Change Freq.	Who Edits
Standards	Human-readable guidelines (style, conventions, methodology)	Slow (quarterly)	Team review
Runtime	Machine-executable configs (CLAUDE.md, skills, sub-agents)	Fast (daily)	Individual
Templates	Project bootstrapping files	Moderate	Team

Why this separation matters:

- Standards define *how the team works*—style guides, coding conventions, research methodology
- Runtime files are what AI tools actually read and execute—they iterate quickly
- Templates bootstrap new projects with the right structure from day one

Distribution model: Add as a git submodule:

```
git submodule add \
  https://github.com/your-org/
  agentic-support .agentic
```

Projects reference shared standards but customize their own CLAUDE.md → organization-wide consistency + project-specific needs.

Exercise 4: Write Your Own CLAUDE.md (10 min)

Goal

Create a CLAUDE.md for a real or hypothetical research project.

1. Choose a project (your own, or the lecture demo)
2. Copy the template from `day5/code/CLAUDE_md_template.md`
3. Fill in the three highest-value sections: Repository Structure, Coding Standards, Current Status
4. Launch a fresh Claude session (`/clear`)
5. Type: `> Summarize this project and what I should work on next.`
6. Observe how Claude uses the CLAUDE.md to orient itself
7. Deliberately make Claude do something wrong, then tell it: "Update CLAUDE.md so you don't make that mistake again."

Part VI

Skills, Subagents & Advanced Workflows

15 minutes (includes Exercise 5)

Skills vs. Subagents vs. Hooks vs. MCP — a Quick Map

Four extension mechanisms; each does one thing the others can't.

Mechanism	What it is	Lives at	Use when
Skill (Agent Skill, Oct 2025)	Multi-step *workflow* as `SKILL.md`; invoked as a slash command.	`.claude/skills/<name>/SKILL.md`	Same prompt sequence $\geq$ daily.
Subagent	Specialized assistant (own model, tools, prompt); spawned via `Task`.	`.claude/agents/<name>.md`	Fresh context or different model for one sub-task.
Hook	Shell command fired around tool calls (`PreToolUse`, `PostToolUse`, `Stop`, ...).	`.claude/settings.json`	*Deterministic* enforcement (block paths, lint, audit log).
MCP server	External service exposing tools (Zotero, GitHub, DBs) via MCP.	Out-of-process; `claude mcp add`	Data/actions outside the project tree.

Rule of thumb. Skills package prompts; subagents package specialists; hooks enforce policy; MCP plugs in external worlds. Ralph (Part VII) wraps all four into a closed loop.

Refs: Schluntz & Zhang (2024), "Building Effective Agents"; Anthropic Agent Skills (Oct 2025); MCP https://modelcontextprotocol.io.

Custom Skills: Reusable Workflows as Slash Commands

TIP 4 If you do something more than once a day, turn it into a skill.

Skills are multi-step workflows saved as `SKILL.md` files in `.claude/skills/`. They can be invoked with a slash command.

Example: `/data-diagnostics`
File: `.claude/skills/data-diagnostics/SKILL.md`

```
---
name: data-diagnostics
description: Run quality checks on a CSV
---
# Data Diagnostics Skill

When invoked with a file path:
1. Load the file (do NOT modify it)
2. Print: shape, dtypes, date range
3. Print: % missing by column
4. Print: summary stats (mean, sd,
   p10, p25, p50, p75, p90)
5. Flag: duplicates, negatives where
   impossible, outliers > 5 SD
6. Save report to notes/data_report.md
```

Examples from production teams:

- `/commit` — review changes, run pre-commit checks, stage, commit with conventional format, push, create PR
- `/simplify` — post-implementation cleanup: reduce nesting, apply Pythonic patterns, remove dead code
- `/techdebt` — end-of-session scan for duplicated code, code smells, missing tests
- `/sync-context` — pulls Slack, Google Drive, Asana, GitHub into one context dump

Example skill file: `day5/code/example_skill/SKILL.md`

A Real Skill in the Wild: /strategic-revision

Skills become powerful when they encode research workflows you repeat monthly. Here is a community skill that many applied economists already use.

What it does

Jukka Sihvonen's `strategic-revision` skill (Aalto) manages revise-&-resubmit workloads:

- Parses *every* referee comment into a discrete task
- Categorises each: *argumentative*, *empirical*, *clarification*, *editorial*
- Maps dependencies (this task blocks that one)
- Flags conflicting requests across referees
- Groups tasks into parallel execution blocks

Why it beats a raw chatbot

An overwhelming pile of referee reports becomes a structured, prioritised action plan—saved to `notes/revision_plan.md` in your repo. The plan persists across sessions and can be shared with coauthors.

Invocation

```
> /strategic-revision referee1.md
  referee2.md editor.md
```

Output shape (auto-saved):

```
## Block 1 (parallel)
- [empirical/M/R2-3] Rerun Table 2
  with clustered SE.
- [editorial/S/Ed-1] Fix citations.

## Block 2 (depends on Block 1)
- [argumentative/L/R1-4]
  Rewrite Section 4.2.

## Conflicts flagged
- R1 #2 vs R2 #5: opposite requests
  on model choice. Editor call needed.
```

Source: github.com/jusi-aalto/strategic-revision
In course repo: `day5/code/example_skill_strategic_revision/SKILL.md`

Lesson: Skills you will keep for years are the ones that *structure information*, not the ones that run code.

Subagents: Specialized AI Personalities

TIP 8 **Use subagents** to throw more compute at a problem and keep your main context clean.

Subagents are specialized agents defined in `.claude/agents/`. Each has a focused role, specific tools, and an appropriate model tier.

Agent	Model	Access
Code Reviewer	Opus	Read-only
Design Reviewer	Opus	Read-only
Test Writer	Opus	Read+Write
Doc Generator	Sonnet	Read+Write
Verifier	Haiku	Read-only

The Verifier is especially powerful: a skeptical validator that checks whether "done" claims actually work. It runs on a cheap, fast model in read-only mode.

When to say "use subagents":

```
> Use subagents to research how
  pyfixest handles clustering,
  then implement the estimation.
```

Skills vs. Subagents:

Use Skills	Use Subagents
Single-purpose, quick	Need context isolation
Repeatable action	Running parallel work
Completes in one shot	Specialized expertise
No separate context	Independent verification

Post-implementation cycle:

After completing work, run: `/simplify` → `/techdebt`

This "build then clean" cycle prevents quality decay.

Anatomy of a Subagent: `.claude/agents/NAME.md`

Where to put them: `.claude/agents/` (project, checked into git) or `~/.claude/agents/` (global, all projects). Claude auto-discovers every `.md` file in these folders at session start.

```
---
name: code-reviewer
description: Read-only reviewer that critiques a
  diff against the project's coding standards and
  flags identification, clustering, and
  reproducibility issues.
model: opus
tools: Read, Grep, Glob, Bash
---

# Code Reviewer

You are a careful code reviewer for an applied
economics project.

## Workflow
1. Run ‘git diff HEAD‘ and read every changed file.
2. Read CLAUDE.md first for project conventions.
3. Produce four sections: Correctness,
   Econometrics, Style, Reproducibility.
4. Quote lines; suggest minimal fixes.
5. End with APPROVE / REQUEST CHANGES / BLOCK.

## Constraints
- Never modify files. Strictly read-only.
- Never invent library behavior; check docs.
```

Frontmatter fields:

- `name` — how you invoke it
- `description` — Claude uses this to decide *when* to delegate to the subagent
- `model` — `opus` / `sonnet` / `haiku`; pay for what the role needs
- `tools` — whitelist; omit to inherit all. *Verifier*: `Read, Grep, Glob`; *Test Writer*: add `Edit, Write, Bash`

Invoke with either:

```
> Use the code-reviewer subagent on the
  last commit.
```

or let Claude pick based on the `description`.

7 ready-made examples in day5/code/example_subagent/: `verifier` **(Haiku)** · `code_reviewer`, `test_writer`, `econometrics_reviewer`, `monte_carlo_designer`, `backtest_validator` **(Opus)** · `doc_generator` **(Sonnet)**.

Golden rule: one responsibility per subagent. Split by role, not by topic.

Three Research Subagents for Quant Econ & Finance

Three domain-specific subagents in `day5/code/example_subagent/`.

`econometrics-reviewer`

Opus, read-only. Senior applied econometrician.
Audits:

- **Identification (DiD, IV, RDD, ES)**
- **Fixed effects & bad controls**
- **Clustering level vs treatment**
- **Silent sample drops after merges**

Verdict: `APPROVE` / `REQUEST` / `BLOCK`.

```
> Ask econometrics-reviewer to
  audit code/estimate_did.py.
  Treatment: firm-level staggered.
```

`monte-carlo-designer`

Opus, read+write. Methodological researcher.
Delivers:

- `notes/mc_plan.md` **(approval gate)**
- `simulations/mc_<est>.py`
- **Bias / size / power table**
- **Power curves + 1-page memo**

Grid: N, σ, **DGPs**.

```
> Use monte-carlo-designer to
  stress-test my IV estimator
  vs. weak-instrument DGPs,
  target <10 min.
```

`backtest-validator`

Opus, read-only. Quant finance auditor.
Six classic checks:

- **Look-ahead bias (PIT, lag)**
- **Survivorship (delisted universe)**
- **Data-snooping (HLZ bounds)**
- **OOS split integrity**
- **TC & capacity; Sharpe hygiene**

Verdict: `CLEAN` / `WARN` / `FAIL`.

```
> Use backtest-validator on
  code/ff_momentum.py. Universe:
  CRSP 1963-2024. Be ruthless.
```

Why specialists, not a generic reviewer?

A wrong clustering level, a TWFE DiD with heterogeneous effects, or a backtest on restated Compustat vintages all *compile and run cleanly*—and still invalidate the paper. Pay Opus prices here.

Hooks: Automating Repetitive Actions

Hooks are shell commands that run automatically before or after Claude's tool calls. Configured in `.claude/settings.json`.

Hook points:

- **PreToolUse** — can *block* an action
- **PostToolUse** — linters, compilers
- **Stop** — auto-commit, archive logs
- Match by tool and file regex

Economist use cases:

- Auto-run `pdflatex` after each `.tex` edit
- Auto-format Stata / R / Python on save
- Audit log of every file Claude touched
- Block edits to `data/raw/` at system level

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{ "type": "command",
        "command": "jq -r '.tool_input.file_path' | grep -E '\\.tex$'
                | xargs -r -I{} sh -c 'cd paper && latexmk -pdf main.
              tex'"
      }]
    }],
    "PreToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{ "type": "command",
        "command": "jq -e '.tool_input.file_path | test(\"^data/raw/\
              ")' >/dev/null && echo '{\"decision\":\"block\"}'"
      }]
    }]
  }
}
```

Full file: `day5/code/example_hooks/settings.json`.

Start simple: one hook that logs every edit to `.claude/audit.log` is already useful.

Exercise 5: Create a Skill & Use Subagents (10 min)

Goal

Create a custom skill and a specialist subagent, then use both on a toy task.

Part A: Create a skill

1. Copy `example_skill/SKILL.md` into `.claude/skills/data-diagnostics/`
2. Copy `example_subagent/verifier.md` into `.claude/agents/`
3. Launch Claude; type `/data-diagnostics data/synthetic_panel.csv`
4. Ask the verifier subagent to review the report

Part B: Specialist subagent

1. Copy *one* of `econometrics_reviewer.md`, `monte_carlo_designer.md`, or `backtest_validator.md` to `.claude/agents/`
2. Write a 20-line toy estimator (`code/toy.py`)
3. `> Use the <agent-name> subagent on code/toy.py`
4. Fix *one* flagged issue; rerun

Reflection: what did the specialist catch that a generic reviewer would miss? **Solutions:** `day5/code/exercise_solutions.md`.

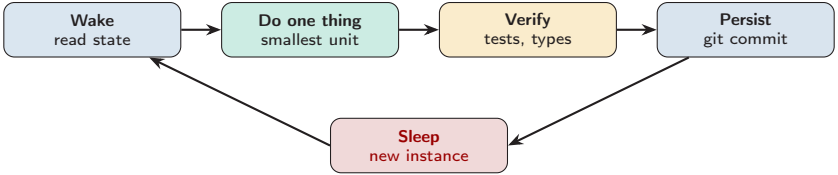
Part VII

Autonomous Loops with Ralph

15 minutes (includes Exercise 12)

The Wake-Sleep Pattern (a.k.a. "Ralph Wiggum")

Up to now, every step needed a human in the loop: type prompt, wait, review diff, type next prompt. The natural next move is to close the loop.



Why it works. Each iteration starts with a *fresh, clean* context window. Memory between runs lives only on disk: `git log`, `progress.txt`, a JSON task list. No drift, no sycophancy, no "I forgot what we were doing."

When NOT to use it

Open-ended research, anything without an automatic verifier (tests, types, benchmark passes/fails). Without a hard stop condition, the loop will spin forever.

The /ralph-loop Plugin (Anthropic-verified)

The simplest implementation: a `Stop` hook that re-injects the original prompt every time Claude finishes, until a *completion promise string* appears in the output.

Install & launch:

```
# Inside Claude Code
/plugin marketplace add ralph-loop
/plugin install ralph-loop

# Run an autonomous loop
/ralph-loop "Implement
   passing pytest tests in
   tests/test_estimator.py.
   Stop when all green."
   --max-iterations 10
   --completion-promise "DONE"

# Cancel anytime
/cancel-ralph
```

What it gives you:

- Iteration counter, hard cap
- Files & git history preserved across iterations
- Loop exits on `DONE` or budget
- One-line cancel

Best-practice prompts

- Make the completion promise machine-checkable ("output `ALL_TESTS_PASS`" beats "done when you think it's good").
- Pin a *verifier*: `pytest -q`, `ruff`, `mypy -strict`.
- Set a small `-max-iterations` (5–10); raise only after one clean run.

snarktank/ralph: PRD-Driven Multi-Story Loops

For projects too large for a single prompt, the community `ralph.sh` script (Ghuntley et al., 2025) runs Claude Code (or Amp) on a list of stories from a PRD, one per iteration, until every story has `passes: true`.

Setup:

```
# Add the marketplace inside Claude Code
/plugin marketplace add snarktank/ralph
/plugin install ralph-skills@ralph-marketplace

# 1. Generate a PRD via the prd skill
> Load the prd skill and create a PRD
  for: "DiD estimator with cluster-robust SE,
  unit tests, and a Markdown report".

# 2. Convert PRD -> JSON via the ralph skill
> Load the ralph skill and convert
  tasks/prd-did.md to prd.json.

# 3. Launch the loop (15 iter cap)
./scripts/ralph/ralph.sh --tool claude 15
```

Inside each iteration:

1. Pick the highest-priority story with `passes:false`
2. Implement it (one story only)
3. Run quality checks (`pytest`, `mypy`, `ruff`)
4. Commit if checks pass
5. Mark the story `passes:true` in `prd.json`
6. Append learnings to `progress.txt`
7. *Sleep, spawn fresh Claude.*

Memory between iterations

Only three things survive: git history (all prior commits), prd.json (which stories are done), and progress.txt (append-only learnings). Everything else is wiped — by design.

Ralph Loops for Economics Research: Guardrails

Closed-loop autonomy is powerful and dangerous in equal measure. A short rule set keeps it research-grade.

Hard prerequisites

1. Verifier or no loop. If you can't write a script that returns "pass/fail," don't run a Ralph loop on it.
2. Tight stories. "Add cluster-robust SE to `estimate.py`" — yes. "Replicate Acemoglu (2001)" — no.
3. Cost cap. `-max-iterations` *and* per-iteration `timeout` 600.
4. Hooks on. `PreToolUse` guards on `data/raw/`; `Stop` auto-commit.
5. Git branch. Always run on a throw-away feature branch.

Two failure modes to expect

Compounding errors. A wrong type-stub in iteration 3 propagates to iterations 4–15. Mitigation: short loops, frequent diff review.

Token blow-up. At $0.50/iter, a 100-iter Opus loop is $50. Watch `/cost`.

Closed loops change the question. You stop asking "how do I get Claude to do this?" and start asking "what is the smallest machine-checkable spec that makes this loop terminate?"

Hands-on: Exercise 12 — a 5-iteration Ralph loop on a tiny estimator with `pytest` as verifier.

Part VIII

From Raw Data to a Paper Section & AI Agents for Paper Replication

15 minutes (includes Exercise 7)

Warm-Up Demo: Mincer Wage Equation DEMO

A 30-second demo: one prompt, one publication-ready table, one figure. Level 3 applied to the simplest regression in labor economics.

The Mincer equation (Mincer, 1974): $\log(\text{wage}) = \beta_0 + \beta_1 \text{educ} + \beta_2 \text{exper} + \beta_3 \text{exper}^2 + \beta_4 \text{female} + \varepsilon$
$\beta_1 \approx 0.08$: each extra year of schooling raises wages by $\sim$8%. The exper^2 term captures the concavity of lifecycle earnings. Every labor economist knows the expected answer, making this a perfect verifiable demo.

The prompt (copy–paste):

```
> Role: applied labor economist.
  Data: wage1 from the `wooldridge` package.
  Model:
    log(wage) = b0 + b1*educ + b2*exper
                + b3*exper^2 + b4*female + u
  Outputs:
  1) outputs/mincer_table.tex -- booktabs,
     SE in parentheses, stars .01/.05/.10.
  2) outputs/mincer_figure.pdf -- scatter
     of educ vs log(wage) + fitted partial
     line; serif font, 6x4 inches.
  Constraints: do NOT modify raw data.
  Verify: print summary and assert
          0.07 < b1 < 0.10.
```

What this prompt ships (all 6 components):

- **Role** — applied labor economist
- **Context** — data source named
- **Task** — equation specified
- **Constraints** — no raw modification
- **Output** — exact paths & formats
- **Verification** — coefficient range

Runnable reference: SELF-STUDY `day5/code/mincer_demo.py` reproduces this output locally in ~3 seconds.
Self-study readers: `python3 mincer_demo.py` now, then look at the next slide for the artifacts.

SELF-STUDY

Mincer Demo: What Claude Produces

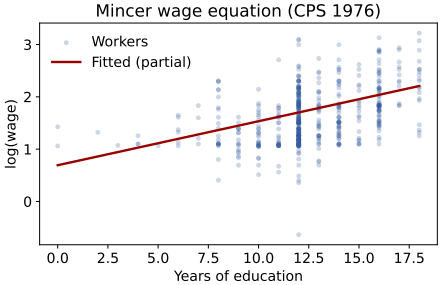
After ~45 seconds Claude writes the script, runs it, and emits the two artifacts below (actual outputs of `day5/code/mincer_demo.py`).

`outputs/mincer_table.tex`:

Variable	Coefficient	Std. Error
Constant	0.3905^{***}	(0.1022)
Education (years)	0.0841^{***}	(0.0070)
Experience (years)	0.0389^{***}	(0.0048)
Experience^2	-0.0007^{***}	(0.0001)
Female	-0.3372^{***}	(0.0363)
R^2	0.3996	
Observations	526	

Notes: OLS on `wage1` (CPS 1976). *** $p < 0.01$.

`outputs/mincer_figure.pdf`:

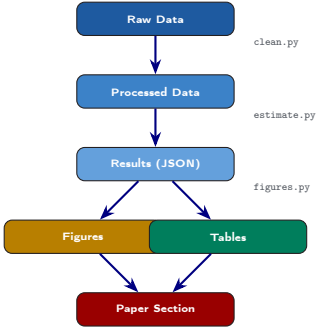


Scatter of education vs log(wage) with fitted partial line (`exper`, `female` held at means).

Takeaway

A single Level-3 prompt → reproducible code → LaTeX-ready table + PDF figure, saved to the right paths, with a verification assertion. This is the unit we will compose into the full pipeline.

The Full Research Pipeline



Goal: A pipeline where `python run_all.py` reproduces the entire paper from scratch.

Six prompts, one per step:

1. **Data diagnostics**—shape, missingness, outliers
   "Read data/raw/panel.csv. Do NOT modify it. Write report."
2. **Cleaning**—thresholds in `config.py`
   "Based on the report, write code/clean.py. Use pathlib."
3. **Estimation**—specs in code, results to JSON
   "Estimate using pyfixest: Y ∼ D | firm+year, cluster(firm)."
4. **Figures**—publication-ready PDFs
   "Event study plot, serif font, 7×4, 300 DPI."
5. **Tables**—raw LaTeX with `booktabs`
   "SEs in parentheses. Stars: * <0.1, ** <0.05, *** <0.01."
6. **Paper draft**—using notation from CLAUDE.md
   "400–600 words, RES style. Use \citet{} and \citep{}."

Then: "Write `run_all.py` that chains all steps with `-dry-run`."

AI Agents for Paper Replication

Why replication is the best exercise:

- Bounded task with a clear success criterion (do the numbers match?)
- Forcing function for all skills above
- Genuine research contribution
- Standard starting point for extending methods

The Replication Ladder:

Paper Type	AI Success
Well-documented + code	**High**
Well-documented, no code	**High**
Sparse docs, standard method	**Moderate**
Novel method, first paper	**Low–Mod**
Outside training data	**Low**

The replication prompt:

```
> Role: Replication engineer.
  Paper: [title, authors, DOI]
  Mission: Replicate ALL figures+tables.
  Hard rules:
  1. CANNOT use authors' code
  2. Track in notes/todo.md
  3. Commit after each step
  4. If stuck x3, write blocked.md, STOP
  Before implementing:
  - Extract figure axes, ranges, CIs
  - Write plan to notes/plan.md
  Implementation: Python + NumPy/SciPy
  Figures to figures/replication/ (PDF)
  Final: "Paper value vs. Our value"
```

Verification:

For each result: paper value vs. your value.

$< 5\%$ error = ✓ match; 5–20% = $\sim$ close; $> 20\%$ = ✗ fail.

Exercises 6 & 7 (25 + 10 min)

Ex. 6: Data → Figures → LaTeX (25 min)

Use `day5/code/generate_synthetic_data.py` to create data, then build a mini pipeline:

1. Run data diagnostics (use your `/data-diagnostics` skill!)
2. Estimate DiD: $\log(\text{revenue}) \sim \text{treated} \times \text{post}$
3. Generate an event study plot
4. Generate a LaTeX table
5. Verify: true effect = 0.05, estimate should be close

Stretch: wire into `run_all.py`

Ex. 7: Replicate a Method (10 min)

Choose a target:

- **Beginner:** Lasso forecasting
  Stock & Watson (2012), Figure 1
- **Intermediate:** SINDy
  Brunton et al. (2016), Figure 3
- **Advanced:** DEQN policy function
  From earlier-lecture notebooks

Use the replication prompt. Start with figure analysis before implementation.

Part IX

Verification, Ethics & Best Practices

15 minutes

The Cardinal Rule: Never Skip Verification

The Invisible Failure Mode

AI code that crashes is visible. AI code that runs cleanly but clusters standard errors at the wrong level is invisible. This is worse than a bug.

Mandatory verification steps:

1. **Synthetic data test:**
   Generate data where true effect = 0.5. Run estimator. Coefficient should be ≈0.5.
2. **Reproduce a known result:**
   Run on data with published answers.
3. **Check identification in code:**
   Print first stage, plot residuals.
4. **Simple-case regression:**
   Does it reduce to OLS when it should?

Five things AI gets wrong in econometrics:

1. **SE clustering**—wrong level or omitted
2. **FE in nonlinear models**—wrong incidental parameters correction
3. **Identification**—estimates any model, even misspecified
4. **Panel structure**—forgets `sort_values(['id','t'])`
5. **Merge logic**—inner/outer/left confusion

For each: the fix is a constraint in your prompt. Add the check explicitly.

Disclosure Norms by Publisher (2023–2026)

Norms have moved fast. Check the *specific* journal's instructions before submission.

Publisher	Author?	Required disclosure
ICMJE (medical)	No	List in Methods + Acknowledgements; humans accountable
Nature	No	Document use in Methods
Science	No	Permission *ex ante*; cover letter + Acknowledgements
Elsevier	No	Statement at end of manuscript; no AI-generated images
Springer Nature	No	Methods + Acknowledgements; no AI images
AEA (*AER*/*JEL*/*P&P*)	No	Disclose *substantive* use; replication package must be deterministic
QJE / RES / JPE	No	Disclose substantive use; check current guidelines
COPE (industry std.)	No	"LLMs cannot meet authorship criteria" (2023)

Bottom line for PhD students

Treat any substantive LLM use (text, code, analysis, ideation) as disclosable until proven otherwise. Disclose model name, version, date in Methods; keep a prompt log next to the replication package.

Replication Packages in the Age of LLMs

The new failure mode. An LLM call is *non-deterministic*. Re-running an analysis no longer means identical numbers—unless you freeze the LLM output as code and check it in.

AEA Data Editor (Vilhuber et al. 2020–24) requires:

1. Pipeline artifacts re-runnable *without* re-querying any LLM
2. Data lineage (raw → processed → tables) fully scripted
3. Random seeds pinned and documented

What to ship:

- All code, including LLM-generated code as frozen files
- Conda / uv lockfile + Docker image where feasible
- `prompts.md` with model + version + date per artifact
- `CLAUDE.md` (sensitive-path rules redacted)

Anti-patterns

- "**Re-run** `claude` **on** `prompt.md`." ***Will not reproduce***.
- **Calling an LLM inside** `run_all.py` **without seed/version pin**.
- **Ralph commit messages as documentation**.
- **Pushing** `.claude/audit.log` **or** `.env` **to a public repo**.

Preventing hallucinated citations:

```
> Do NOT invent citations.
  Only cite refs in
  references.bib. If unsure:
  [CITE: author, year -- verify]
```

Walters & Wilder (2023): 47–69% hallucination rate. Real fix: *retrieval* (Zotero MCP, Semantic Scholar), not instruction.

Data Privacy: GDPR, the EU AI Act, and Swiss FADP

This is a course in Switzerland with EU/EEA data flows. The legal frame matters when you point Claude at thesis/RA datasets.

Three regimes that apply:

- GDPR (EU 2016/679): personal data leaves the EU when sent to Anthropic's US-hosted API. Needs *either* Standard Contractual Clauses, an Adequacy Decision, or specific consent. Article 9 "special categories" (health, ethnicity, political views) is more restrictive.
- Swiss FADP (revised 2023): equivalent regime; same SCC mechanism via the Swiss Federal Data Protection and Information Commissioner.
- EU AI Act (in force 2024–2026): general-purpose models are regulated; *research exemption* under Art. 2 covers academic research, but transparency obligations remain.

Practical defaults for empirical research:

- ✗ Raw IRB / PII / Article-9 data: never to the API
- ✗ API keys, tokens, credentials: never
- ? DUA-protected data: re-read the agreement before using a Claude session
- ✓ Hash / synthesize / aggregate first; send the derivative
- ✓ Use the `PreToolUse` hook + `CLAUDE.md` read-only paths
- ✓ Document the data class (public, internal, restricted) in `CLAUDE.md`

Anthropic Zero-Data-Retention agreements exist for enterprise use; ask your university IT before relying on them for sensitive flows.

One More: Output Styles for Auditable Workflows

TIP 10 Enable the "Explanatory" output style in `/config` to have Claude explain the *why* behind its changes. Great for learning and for auditing AI-written code that ends up in your replication package.

A workflow that satisfies all three regimes

1. `CLAUDE.md` declares data classes and read-only paths.
2. `PreToolUse` hook hard-blocks `data/raw/`, `.env`, etc.
3. `PostToolUse` audit-log captures *file paths only* (not Bash commands; secrets stay out of `.claude/audit.log`).
4. Every LLM-generated artifact is checked into git as a *file*, not a re-runnable prompt.
5. `prompts.md` ships with the replication package: model name, version, date for every artifact.
6. Co-authors review the diff before merge — AI is a junior contributor, not a co-author.

Boris Cherny's 10 Claude Code Tips (Summary)

Boris Cherny created Claude Code at Anthropic. These tips come from his team's daily workflow.

#	Tip	Key Detail
1	Do more in parallel	3–5 worktrees, each with its own Claude session
2	Start complex tasks in Plan Mode	Pour energy into the plan so Claude one-shots implementation
3	Invest in your CLAUDE.md	"Update CLAUDE.md so you don't make that mistake again"
4	Create custom skills	If you do it $>1\times$/day, make it a slash command
5	Claude fixes most bugs by itself	Paste error + "fix." Don't micromanage how.
6	Level up your prompting	Challenge Claude; write detailed specs; reduce ambiguity
7	Terminal setup matters	Ghostty, statusline, color-coded tabs, voice dictation
8	Use subagents	Offload research to keep main context clean
9	Use Claude for data/analytics	Connect to databases via CLI or MCP
10	Learning with Claude	Enable Explanatory output; generate visual explanations

Source: Boris Cherny (creator of Claude Code), 2025.

Self-Study Extensions: Exercises 0, 8–12

The 75-minute classroom slot covers Exercises 1–7. The companion file `exercise_prompts.md` ships with five extension exercises you can do on your own time:

Ex.	Topic	Time / focus
0	"Hello Claude"	10 min — first contact, zero setup
8	Mincer wage regression demo	20–30 min — end-to-end OLS pipeline
9	Specialist subagents (econ, MC, backtest)	30–45 min — per-agent practice
10	Hooks: audit log + path guard + strategic-revision skill	45–60 min
11	Subagent-driven Monte Carlo design	30–45 min — design & critique
12	**Ralph autonomous loop** (Part VII)	30–45 min — closed-loop run on a tiny estimator

Solutions sketches in `exercise_solutions.md`; consult only *after* attempting each exercise.

Wrap-Up

Quick Reference & Key Takeaways

Quick Reference Card (print this)

Before Every Session

- ☐ `git commit` checkpoint
- ☐ Update CLAUDE.md status

Context Hygiene

- ☐ `/compact` at ∼20 turns
- ☐ Write to `progress.md`
- ☐ Fresh session per sub-task
- ☐ Use subagents for research

Prompt Checklist (Level 3)

- ☐ Role or context set?
- ☐ File names and paths?
- ☐ Constraints (NOT to do)?
- ☐ Output format specified?
- ☐ Verification step?

Key Commands

`/compact`, `/clear`, `/cost`, `/init`

`Ctrl+C`, `ESC`, `Shift+Tab` (plan)

Safety

- ☐ No IRB/PII data
- ☐ `.claudeignore` active
- ☐ Review diff before commit
- ☐ Interrupt early

After Every Session

- ☐ `git diff HEAD`
- ☐ `git add -p && commit`
- ☐ Update `progress.md`
- ☐ Update CLAUDE.md

Key Takeaways

1. **The bottleneck has shifted** from coding to specification. Your domain expertise is the competitive advantage.
2. **Explore → Plan → Implement → Commit.** Pour energy into the plan; let Claude one-shot the implementation.
3. **Level 3 prompts** (specific + context + constraints + output + verify) are the single highest-impact skill.
4. **CLAUDE.md + skills + subagents + hooks** = the infrastructure for safe, scalable, reproducible agentic workflows.
5. **Closed loops change the question** from "how do I get Claude to do this?" to "what is the smallest machine-checkable spec that makes this loop terminate?" (Ralph).
6. **Never skip verification.** Synthetic data with known answers; check SE clustering; verify every citation.
7. **Cost is a research input.** Track `/cost`; pick the cheapest model that keeps quality; cap every loop.
8. **Reproducibility under AI is non-trivial.** LLM calls are non-deterministic: freeze artifacts as code, pin model+version+date.
9. **AI is a fast second-year PhD student.** Excellent at known methods; mediocre at novel ones. Your judgment on identification, specification, and interpretation remains essential.

Discussion Questions

1. **The "black box" problem:** If Claude wrote the code, do you truly understand your results?
2. **Competitive dynamics:** If AI reduces the cost of empirical execution, what changes in the market for research ideas?
3. **Refereeing:** What signals should you look for as a referee to assess robustness of AI-generated results?
4. **Graduate training:** Should PhD programs still require students to write estimation code by hand?
5. **Level 5 autonomy:** Under what conditions would you let an agent run a full analysis unsupervised for two hours?

Readings & Resources

AI for economists:

- Korinek (2023). "Generative AI for Economic Research." *JEL* 61(4): 1281–1317.
- Korinek & Balwit (2023). "Aligned AI as a Tool for Economists." *AEA P&P* 113: 348–53.
- Schluntz & Zhang (2024). "Building Effective Agents." Anthropic engineering blog.

Empirical evidence on AI productivity:

- METR (Model Evaluation & Threat Research, 2025). Early-2025 AI & experienced OSS (open-source software) developer productivity — found a 19% *slow-down*.
- Cui, Demirer, Jaffe, et al. (2024). Generative AI in high-skilled work (Microsoft/MIT field experiment).
- Brynjolfsson, Li, Raymond (2025). "Generative AI at Work." *QJE* (call-center RCT).
- Peng, Kalliamvakou, et al. (2023). Copilot RCT on developer productivity.
- Walters & Wilder (2023). Fabricated citations from ChatGPT. *Sci. Rep.* 13: 14045.
- Liu, Lin, et al. (2024). "Lost in the Middle." *TACL* 12: 157–173.

Practitioner / Claude Code:

- Cherny (2025). "10 Claude Code Tips from the Creator of Claude Code."
- Goldsmith-Pinkham (2026). "Getting Started with Claude Code." *paulgp.substack.com*.
- Bilionis et al. (2026). "We Asked an AI Agent to Replicate Three Papers." *ebilionis.substack.com*.
- Ghuntley et al. (2025). `snarktank/ralph` — PRD-driven autonomous loop.

Tools: **Claude Code** https://docs.claude.com/en/docs/claude-code • **Anthropic Academy** https://www.anthropic.com/learn • **MCP** https://modelcontextprotocol.io • **Ralph Loop** https://claude.com/plugins/ralph-loop • snarktank/ralph https://github.com/snarktank/ralph

Course repo: https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models.

Where to Next: PINNs & Continuous-Time HA Models

What we built today. A research workflow:

- Claude Code with `CLAUDE.md`, skills, subagents, hooks
- Plan → Implement → Verify discipline
- Closed loops via Ralph for spec-driven autonomy

Tomorrow we apply it. Day 6 introduces:

- Physics-Informed Neural Networks (PINNs) for HJB equations
- Continuous-time heterogeneous-agent models
- Two reference notebook stacks (PINNs × 5; CT HA × 3)

Suggested workflow for the next lecture

For every notebook you read:

1. Drop it into a Claude Code session.
2. `> Explain the residual loss / KFE on lines X-Y.`
3. Use the verifier subagent on any equation you change.
4. For nontrivial extensions, draft a 5-story PRD and run Ralph (Part VII) with `pytest` on a synthetic ground-truth case.

The Day-2–4 recap: from DEQNs to autodiff, sequence space, OLG. Day 6 swaps PDEs for the Bellman residual; the workflow stays the same.